

Typing Platform Development Roadmap

Phase 1 – Project Setup

Initialize the frontend with React + Vite + TypeScript, Tailwind CSS, shadcn/ui, React Router, Framer Motion, GSAP, Lucide React, Axios, TanStack Query, React Hook Form, Zod, Recharts, and Sonner. Initialize the backend with Express, TypeScript, Prisma, PostgreSQL, JWT, bcrypt, CORS, dotenv, and nodemon. Configure Neon PostgreSQL and Prisma.

Phase 2 – Authentication

Implement Signup, Login, Password Hashing, JWT Authentication, Protected Routes, and Logout. After login, maintain the authenticated user's ID, name, and email.

Phase 3 – Database Design

Create Prisma models: User, Test, PersonalBest, and Achievement with proper relationships and timestamps. The Test model should store duration, mode, WPM, raw WPM, accuracy, errors, consistency, character counts, and creation time.

Phase 4 – Routing

Create placeholder pages for Landing, Login, Signup, Dashboard, Typing Test, Results, Analytics, Leaderboard, Profile, and Settings before implementing the UI.

Phase 5 – Integrate Stitch Designs

Wait for the Google Stitch designs and replace placeholder pages with the generated UI. Avoid styling pages extensively before the final designs are available.

Phase 6 – Build the Typing Engine

Implement the core typing logic: load text, start timer, track keystrokes, compare characters, highlight mistakes, calculate WPM and accuracy, end the test, and prepare results.

Phase 7 – Save Test Data

After every completed test, store duration, mode, WPM, accuracy, errors, consistency, and timestamp in the database using a backend API.

Phase 8 – Dashboard

Display Personal Best, Average WPM, Average Accuracy, Current Streak, and Recent Tests using the stored data.

Phase 9 – Analytics

Build charts and statistics including WPM trend, accuracy trend, practice heatmap, weekly/monthly progress, total tests completed, average mistakes, and personal bests.

Phase 10 – Leaderboard

Implement leaderboards with rankings based on Highest WPM, Highest Accuracy, Most Practice, Weekly, Monthly, and All-Time categories.

Phase 11 – Polish

Add Framer Motion animations, GSAP background effects, loading animations, hover effects, skeleton loaders, toast notifications, and polished empty states.

Recommended Antigravity Workflow

Use focused prompts instead of asking Antigravity to build the whole application. Progress through setup, backend, database schema, UI implementation, typing engine, analytics, and final polish one feature at a time.

Development State Machine

Model the typing session as: Idle → Countdown → Typing → Completed → Save Results → Analytics Updated. This keeps timer logic and state transitions organized and minimizes bugs.